Universidade Federal de Roraima
Programa de Pós-graduação em Computação Aplicada
Arquitetura e Organização de Computadores

Laboratório — Projeto SENTINELA: do circuito digital ao IP Core

Ferramenta: Logisim Evolution (versão 3.8 ou superior)
Modalidade: individual ou em dupla (a mesma dupla do PTT, preferencialmente)

---

1. Cenário

Um gerador a diesel alimenta o laboratório de uma escola em comunidade ribeirinha. Ele não tem supervisão permanente e já queimou duas vezes por superaquecimento — numa delas, o sensor de temperatura estava com fio solto e a leitura "congelou" em um valor normal.

Você foi contratado para projetar o **SENTINELA**, um nó sensor de borda que:

1. lê a temperatura do bloco do motor a cada ciclo de amostragem;
2. **filtra** as leituras para não reagir a ruído elétrico do próprio gerador;
3. **detecta falha do sensor** (valor fora de faixa ou leitura congelada);
4. executa uma **sequência de intertravamento** segura: ligar ventilação → soar alarme → cortar o gerador → exigir rearme manual;
5. mede **quantos ciclos de relógio** gasta por amostra, para que a equipe saiba se o circuito cabe no orçamento de tempo real.

O SENTINELA será construído em quatro etapas, cada uma empacotada como **subcircuito reutilizável** — exatamente a disciplina de projeto de um IP Core.

> **Aviso para quem nunca mexeu com hardware:** a Etapa 0 é obrigatória para você e opcional para os demais. As Etapas 1 a 3 são o núcleo avaliado. A Etapa 4 é um desafio opcional com bônus, pensado para quem pretende seguir PTT5.

Convenções de projeto (valem para todas as etapas)

Item	Convenção
Largura de dados	8 bits sem sinal para amostras (0–255 ≙ 0,0 °C a 127,5 °C, passo 0,5 °C)
Ponto fixo	Coeficientes em **Q1.7** (8 bits, 1 bit inteiro + 7 fracionários); 1,0 ≙ 128
Relógio	Um único `Clock` global; sinal `rst` síncrono, ativo em nível alto
Nomes de sinais	`snake_case`; use `Tunnel` para `clk`, `rst` e barramentos
Subcircuitos	Um subcircuito por etapa, com pinos de entrada à esquerda e saída à direita; aparência personalizada (*Edit* → *Edit Circuit Appearance*)

Universidade Federal de Roraima
Programa de Pós-graduação em Computação Aplicada
Arquitetura e Organização de Computadores

Item	Convenção
Testes	`Simulate → Test Vector` com arquivo `.txt` por subcircuito; print do resultado anexado
Dúvidas técnicas não resolvidas	marque no relatório com `[VERIFICAR]`, como no catálogo de PTT

Etapa 0 — Aquecimento (opcional; obrigatória para quem não tem base de hardware)

Objetivo: dominar o fluxo mínimo do Logisim Evolution: desenhar, simular, analisar e testar.

Componentes: `Pin`, `LED`, `Button`, `Clock`, portas básicas, `Register`, `Counter`, `Hex Digit Display`.

Faça:

1. Construa um contador de 4 bits que conta de 0 a 9 e volta a 0 (use `Counter` com *Maximum Value* = 9) e exiba no `Hex Digit Display`.
2. Abra *Simulate → Ticks Enabled* e observe o contador; depois use *Simulate → Logging* para gravar a saída.
3. Rode *Project → Analyze Circuit* em uma porta `XOR` de 2 bits e veja a tabela-verdade gerada automaticamente.

Pergunta reflexiva: o `Counter` do Logisim "sabe" contar sozinho. Em que ponto da ementa (lógica combinacional × sequencial) está a fronteira entre o que ele faz e o que uma `ALU` faz?

Não há entrega desta etapa; ela existe para que a Etapa 1 não seja o primeiro contato com a ferramenta.

Etapa 1 — Caminho de dados mínimo: `alu8` e banco de registradores

Objetivo: construir a unidade aritmética que todas as etapas seguintes vão reutilizar, com *flags* de resultado e um par de registradores com escrita controlada.

Componentes do Logisim: `Adder`, `Subtractor`, `Shifter`, portas `AND`/`OR`/`XOR`, `Multiplexer` (4 entradas × 8 bits), `Comparator`, `Register` (8 bits), `Splitter`, `Tunnel`, `Constant`.

Universidade Federal de Roraima
Programa de Pós-graduação em Computação Aplicada
Arquitetura e Organização de Computadores

Especificação funcional — `alu8`

`op[1:0]`	Operação	`y[7:0]`	Flags afetadas
`00`	ADD	`a + b`	`zero`, `carry`
`01`	SUB	`a - b`	`zero`, `carry` (empréstimo)
`10`	AND	`a & b`	`zero`
`11`	SHR	`a >> 1` (lógico)	`zero`, `carry` recebe o bit descartado

Pinos: entradas `a[7:0]`, `b[7:0]`, `op[1:0]`; saídas `y[7:0]`, `zero`, `carry`.

Banco mínimo de registradores — `regs2`

Dois registradores `r0` e `r1` de 8 bits. Entradas: `din[7:0]`, `we` (write enable), `sel` (qual registrador escrever), `clk`, `rst`. Saídas: `r0_out`, `r1_out`. A escrita ocorre na borda de subida de `clk` **somente** se `we = 1`.

Casos de teste obrigatórios

#	`a`	`b`	`op`	`y` esperado	`zero`	`carry`
1	0x0F	0x01	ADD	0x10	0	0
2	0xFF	0x01	ADD	0x00	1	1
3	0x10	0x10	SUB	0x00	1	0
4	0x05	0x0A	SUB	0xFB	0	1
5	0xF0	0x3C	AND	0x30	0	—
6	0x81	—	SHR	0x40	0	1
7	`regs2`: escrever 0xAA em `r1` com `we=1`, depois tentar escrever 0x55 com `we=0`			`r1_out` permanece 0xAA		

Grave esses casos em `test_alu8.txt` (formato *Test Vector*: primeira linha com nomes dos pinos, uma linha por caso) e anexe o print da execução.

Dicas de implementação: - O `Subtractor` do Logisim expõe o empréstimo como saída `b_out`; decida se você quer `carry` com a semântica de "empréstimo" ou de "sem empréstimo" e **documente** — essa escolha reaparece em qualquer ISA real. - Calcule as quatro operações em paralelo e selecione com o `Multiplexer`; é assim que ALUs reais funcionam, e isso facilita a Etapa 4. - `zero` é um `NOR` de 8 entradas sobre `y`, ou um `Comparator` contra `Constant 0`.

Pergunta reflexiva (caminho de dados): sua `alu8` calcula as quatro operações o tempo todo e descarta três. Em termos de **área** e **consumo** (critérios do relatório de síntese de PTT2), o que isso custa? Haveria alternativa para a `SHR` que não usasse o `Shifter`?

Universidade Federal de Roraima
Programa de Pós-graduação em Computação Aplicada
Arquitetura e Organização de Computadores

Etapa 2 — Filtro FIR de 4 taps em ponto fixo e contador de ciclos

Objetivo: reutilizar a `alu8` em uma unidade **MAC** (multiplica-acumula) sequencial que implementa um filtro FIR de 4 taps, e instrumentar o circuito com um contador de ciclos para medir desempenho. O mesmo bloco MAC é o átomo de um acelerador de RNA (PTT2 e PTT3).

Componentes do Logisim: subcircuito `alu8` (Etapa 1), `Shift Register` (4 estágios × 8 bits) ou 4 `Register` em cascata, `Multiplier` (8×8 → 16 bits), `Register` acumulador de 16 bits, `ROM` ou `Constant` para os coeficientes, `Counter` (contador de ciclos), `Counter` de 2 bits (índice do tap), `Bit Selector`/`Splitter` para o truncamento.

Especificação funcional — `fir4`

Equação em ponto fixo (coeficientes em Q1.7):

```
y[n] = ( h0·x[n] + h1·x[n-1] + h2·x[n-2] + h3·x[n-3] ) >> 7
```

Restrição: `h0 + h1 + h2 + h3 = 128` (ganho unitário em Q1.7). Sugestão de partida: `h = {16, 48, 48, 16}` (passa-baixa simétrico). O resultado após `>> 7` cabe em 8 bits; **justifique** no relatório por que (pense no ganho unitário e no pior caso de entrada 0xFF).

Interface: entradas `x_in[7:0]`, `start`, `clk`, `rst`; saídas `y_out[7:0]`, `done`, `ciclos[7:0]`.

Comportamento sequencial esperado (uma amostra por `start`):

Ciclo	Ação
0	`start=1`: desloca `x_in` para dentro do shift register; zera acumulador e índice
1–4	Para `i = 0..3`: `acc ← acc + h_i · x[n-i]` (o somador é a sua `alu8` em modo ADD — pense em como lidar com 16 bits)
5	`y_out ← acc >> 7`; `done=1` por um ciclo

O `Counter` de ciclos deve incrementar enquanto `busy = 1` (entre `start` e `done`) e congelar em `done`; `ciclos` deve ler **5** para esta implementação. Se o seu projeto gastar outro número, está correto desde que o relatório explique por quê.

Casos de teste obrigatórios

#	Sequência de `x_in` (4 amostras, mais antiga primeiro)	`y_out` esperado com `h={16,48,48,16}`	O que valida
1	0x00, 0x00, 0x00, 0x00	0x00	Reset do acumulador
2	0x80, 0x80, 0x80, 0x80	0x80	Ganho unitário em regime
3	0xFF, 0xFF, 0xFF, 0xFF	0xFF	Ausência de overflow no pior caso

#	Sequência de `x_in` (4 amostras, mais antiga primeiro)	`y_out` esperado com `h={16,48,48,16}`	O que valida
4	0x00, 0x00, 0x00, 0xFF	0x1F	Resposta ao impulso (tap `h0`)
5	0x64, 0x64, **0xFF**, 0x64	**≤ 0x80** (calcule o valor exato)	Rejeição de pico de ruído de 1 amostra
6	Qualquer	`ciclos` = 5 e `done` dura exatamente 1 ciclo	Instrumentação

Calcule à mão o valor exato do caso 5 antes de simular e registre-o no `test_fir4.txt`. O ponto do caso é ver que **um pico isolado de 127,5 °C é atenuado** — é isso que impede o alarme falso no cenário.

Dicas de implementação: - Armazene os coeficientes em uma `ROM` de 4 palavras × 8 bits, endereçada pelo contador de tap. Trocar o filtro passa a ser trocar o conteúdo da ROM — essa é a diferença entre um bloco "parametrizável" e um bloco "fixo", vocabulário de IP Core. - O acumulador tem 16 bits, mas a `alu8` tem 8. Você pode (a) construir um somador de 16 bits com duas `alu8` encadeadas pelo `carry`, ou (b) usar um `Adder` de 16 bits direto e reservar a `alu8` para a Etapa 3. Escolha e justifique; a opção (a) é a mais "fiel ao IP Core". - Um pequeno sequenciador de 3 estados (`IDLE`, `MAC`, `DONE`) já resolve o controle. Guarde essa FSM: ela é o embrião da Etapa 3. - Use `Simulate → Logging` para exportar `y_out` por ciclo e comparar com o cálculo em planilha ou Python (esse é o "traço de referência" exigido em PTT9 e nos testes de PTT2).

Pergunta reflexiva (desempenho): seu `fir4` gasta 5 ciclos por amostra com **um** multiplicador. Com quatro multiplicadores em paralelo gastaria 1 ou 2. Estime a razão de área e o *speedup*, e explique por que a Lei de Amdahl limita o ganho quando a aquisição do sensor (E/S) domina o tempo por amostra.

Etapa 3 — FSM de intertravamento com detecção de falha do sensor

Objetivo: projetar e documentar uma máquina de estados finitos **explícita** que consome a saída do `fir4` e comanda ventilação, alarme e corte do gerador, com detecção de sensor defeituoso e rearme manual. Este é o coração de PTT7 e a "unidade de controle" de PTT5.

Componentes do Logisim: subcircuito `fir4` (Etapa 2), `Register` de estado (3 bits), `Comparator` (×2 ou ×3), `Counter` para *watchdog* de leitura congelada, `Register` para guardar a amostra anterior, portas lógicas para próxima-lógica de estado, `Button` (`ack`), `LED` ×3, `ROM` opcional (ver dicas).

Universidade Federal de Roraima
Programa de Pós-graduação em Computação Aplicada
Arquitetura e Organização de Computadores

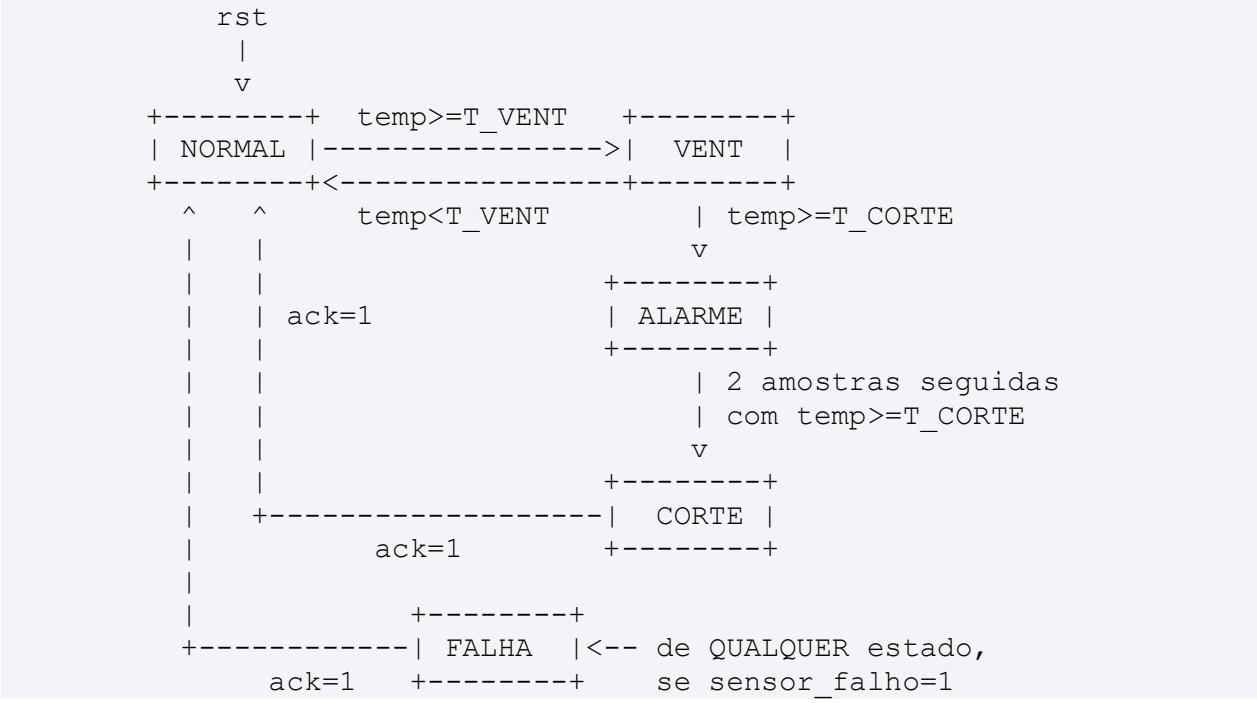
Entradas e saídas do bloco `intertravamento`

Sinal	Direção	Significado
`temp[7:0]`	entrada	saída filtrada do `fir4` (`y_out`)
`amostra_ok`	entrada	`done` do `fir4` — indica nova amostra válida neste ciclo
`ack`	entrada	botão de rearme manual
`clk`, `rst`	entrada	relógio e reset
`vent`	saída	liga ventilação forçada
`alarme`	saída	sirene / LED vermelho
`corte`	saída	desliga o gerador
`falha_sensor`	saída	LED amarelo: sensor suspeito

Limiares (configuráveis por `Constant`)

Nome	Valor padrão	Significado físico
`T_VENT`	0xB4 (90 °C)	acima disso, ventilação
`T_CORTE`	0xD2 (105 °C)	acima disso, corte
`T_FAIXA_MAX`	0xF0 (120 °C)	acima disso, leitura fisicamente implausível → sensor em falha
`N_CONGELADO`	8 amostras	mesmo valor exato por 8 amostras seguidas → sensor congelado

Diagrama de estados



Regras adicionais:

- `sensor_falho = (temp > T_FAIXA_MAX) OR (contador_congelado ≥ N_CONGELADO)`.
- `FALHA` tem **prioridade** sobre qualquer outra transição (é a transição de segurança).
- `CORTE` e `FALHA` só saem por `ack`; nunca voltam sozinhos a `NORMAL`, mesmo que a temperatura caia.
- Transições só são avaliadas quando `amostra_ok = 1` (uma decisão por amostra, não por ciclo de relógio).

Tabela de saídas (Moore)

Estado	Código sugerido	`vent`	`alarme`	`corte`	`falha_sensor`
NORMAL	000	0	0	0	0
VENT	001	1	0	0	0
ALARME	010	1	1	0	0
CORTE	011	1	1	1	0
FALHA	100	1	1	1	1

Observe que `FALHA` corta o gerador: se não confiamos no sensor, o estado seguro é desligado. Discuta no relatório se essa é a decisão correta para o cenário.

Casos de teste obrigatórios

#	Estímulo (sequência de `temp` após o filtro)	Resultado esperado
1	0x60, 0x70, 0x80 (abaixo de todos os limiares)	permanece em NORMAL, todas as saídas 0
2	0x60 → 0xB8 → 0xA0	NORMAL → VENT → NORMAL
3	0xB8 → 0xD5 → 0xD5	VENT → ALARME → CORTE; `corte=1`
4	0xB8 → 0xD5 → 0xA0	VENT → ALARME → VENT (uma única amostra alta não corta)
5	Em CORTE, `temp` cai para 0x40 sem `ack`	permanece em CORTE
6	Em CORTE, `ack=1`	volta a NORMAL
7	`temp = 0xF5` em qualquer estado	vai para FALHA no mesmo ciclo de decisão
8	`temp = 0x90` repetido 8 vezes	FALHA por sensor congelado; com 7 repetições, **não**
9	Em VENT, `temp = 0xF5` e `temp ≥ T_CORTE` simultaneamente	FALHA (prioridade), não ALARME
10	Integração: alimente o `fir4` com 0x64,0x64,0xFF,0x64 (caso 5 da Etapa 2)	o pico isolado **não** dispara VENT

Dicas de implementação: - Comece desenhando a **tabela de transição** completa (estado atual, condições, próximo estado) antes de abrir o Logisim. Ela vai direto para o relatório e é o "modelo formal" que PTT7 exige. - Para o detector de congelamento: um `Register` guarda `temp` da amostra anterior, um `Comparator` testa igualdade, e um `Counter` com *Enable* incrementa se igual e reseta se diferente. - A lógica de próximo estado pode ser feita de duas formas: (a) portas lógicas derivadas da tabela (unidade de controle **hardwired**); (b) uma `ROM` endereçada por `{estado, condições}` cujo conteúdo é o próximo estado e as saídas (unidade de controle **microprogramada**). Implemente a (a) e, se tiver tempo, a (b) como comparação — isso vale na rubrica. - Teste cada transição isoladamente com `Test Vector` antes do teste de integração com o `fir4`. - Estados não utilizados (101, 110, 111): decida o que acontece se o registrador de estado for parar neles (dica: segurança sugere ir para FALHA). Isso é "ausência de estados inalcançáveis/perigosos", propriedade que PTT7 verificará com ferramenta formal.

Pergunta reflexiva (unidade de controle hardwired × microprogramada): compare as duas implementações da lógica de próximo estado. Qual é mais fácil de modificar se a equipe pedir um novo estado `MANUTENCAO`? Qual tende a ter menor latência entre a amostra e a atuação (`corte`)? Relacione com a discussão de Patterson & Hennessy sobre controle hardwired versus microprogramado.

Etapa final — Ponte para o PTT

No relatório, responda obrigatoriamente, em no máximo meia página:

1. **Qual trilha** (PTT2, PTT5, PTT7 ou outra) este laboratório prepara **para você**, e qual dos subcircuitos (`alu8`, `fir4`, `intertravamento`, sequenciador) é o embrião do seu produto principal.

Entregáveis

Item	Descrição	Formato
Projeto Logisim	Um único arquivo com os subcircuitos `alu8`, `regs2`, `fir4`, `intertravamento` (e `sequenciador`, se bônus), mais um circuito `main` que integra tudo com botões e LEDs	`sentinela.circ`
Vetores de teste	Um arquivo por subcircuito, com todos os casos obrigatórios	`test_alu8.txt`, `test_fir4.txt`, `test_fsm.txt`
Evidências	Print do resultado de cada *Test Vector* (todos verdes ou com os falhos justificados) e um print do `main` em simulação com `ciclos` visível	`.png`

Universidade Federal de Roraima
Programa de Pós-graduação em Computação Aplicada
Arquitetura e Organização de Computadores

Item	Descrição	Formato
Relatório	Máximo **2 páginas** (fonte 11): tabela de transição da FSM, respostas às perguntas reflexivas das Etapas 1–3.	`.pdf`
Repositório	Tudo acima em repositório versionado com `README.md` indicando versão do Logisim e como rodar os vetores de teste — mesma exigência-base de todas as trilhas do catálogo	link Git